Business Logic Vulnerabilities in a Web Subscription Application

Payment Process Bypass and Unauthorized Refund — Research and POC Implementation (Track A)

Name: Hiba Kurdieh

ID: 214000820

Institution: Azrieli College of Engineering

Department: Computer Science

1. Vulnerability Description and Theoretical Background

Definition and Theoretical Background: Business Logic Vulnerabilities

Business logic vulnerabilities are flaws in the design and business logic of an application. They occur when the system allows actions that violate intended business rules, such as skipping required steps, performing actions in an incorrect order, or repeating actions that should only occur once. Unlike technical vulnerabilities such as SQL Injection or Cross-Site Scripting (XSS), business logic vulnerabilities may exist even when the code is technically correct.

According to OWASP Top 10 (2021), such weaknesses are categorized under A04:2021 – Insecure Design, which refers to missing or ineffective control mechanisms that enforce business rules.

In web systems, trust boundaries are critical. All security and business decisions must be enforced server-side. Client input cannot be trusted because attackers can modify, replay, or craft requests directly.

Secure workflow enforcement requires state validation and state machines that prevent invalid transitions. If the server allows direct state changes without validation, attackers can bypass intended processes.

Identified Vulnerabilities in This Project

• Vulnerability 1 — Payment Callback Bypass:

The server activates subscriptions based solely on client input (paymentConfirmed: true) without verifying correct state transitions or validating payment with a trusted provider.

• Vulnerability 2 — Refund Abuse Without Validation and Idempotency:

The refund endpoint allows issuing refunds without verifying payment status, subscription eligibility, or whether a refund was already processed.

Impact of the Attack

An attacker can activate subscriptions without paying, issue unauthorized refunds, repeat refund requests multiple times, and cause direct financial loss.

2. Environment and Technologies

Programming Language: TypeScript

Runtime Environment: Node.js

Framework: Express.js

Client: HTML, CSS, JavaScript

Database: In-memory storage

Package Manager: npm

Tools Used for Demonstration

• Web browser

• Browser developer tools (Network panel)

• HTTP requests via application interface

To run the application locally:

1. Install Node.js (v18 or later).

2. Clone the project repository.

3. Run: npm install

4. Start the server using: npm run dev

5. Open the browser at: http://localhost:3000

6. Use the interface to test normal flow and vulnerable endpoints.

3. Proof of Concept (POC)

The application implements the following workflow state machine: REGISTERED → PLAN_SELECTED → PAYMENT_PENDING → ACTIVE.

Attack 1 — Payment Process Bypass

Steps to Reproduce:

1. Register a subscription.

2. Ensure state is PAYMENT_PENDING.

3. Send request:

POST /payment/callback

{ "paymentConfirmed": true }

Result of Exploit

The subscription state changes to ACTIVE without verifying payment with a trusted payment provider.

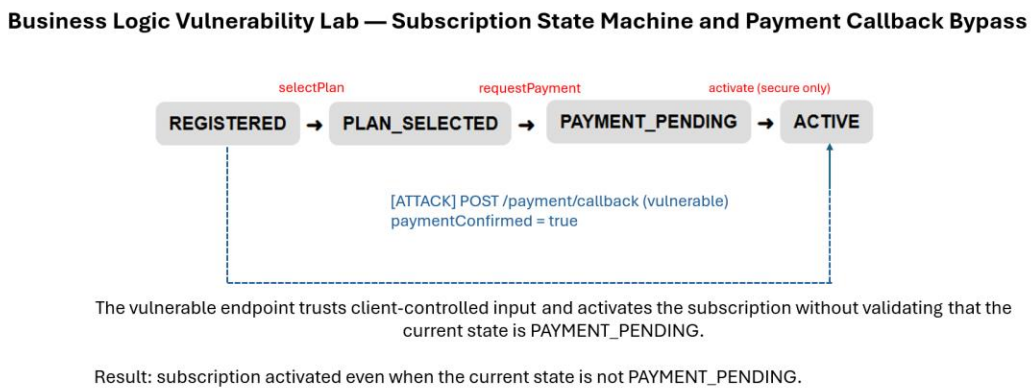

Figure 1 — Subscription state machine and payment callback bypass vulnerability.

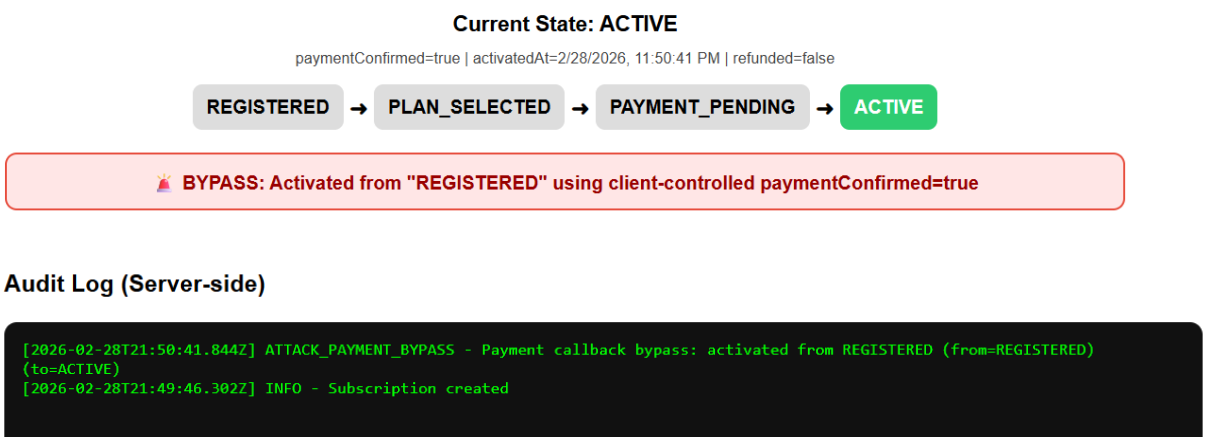

Figure 1a — Payment callback bypass activating subscription directly from REGISTERED state.

Attack 2 — Unauthorized Refund Abuse

Steps to Reproduce:

Send request:

POST /refund

Result of Exploit

The server issues a refund without validating payment status or previous refunds. The request can be repeated multiple times.

Business Logic Vulnerability Lab — Refund Abuse (Secure vs Vulnerable)	
POST /refund (secure) Server validates: • state == ACTIVE • paymentConfirmed == true • refunded == false If checks pass → refund processed If checks fail → request blocked (SECURE_BLOCKED)	**POST /refund (vulnerable)** • Can be executed from ANY state • No state validation • No payment validation • No idempotency check Result: refunded = true even if state is not ACTIVE. Request can be repeated multiple times.

Figure 2 — Refund abuse vulnerability with secure validation comparison.

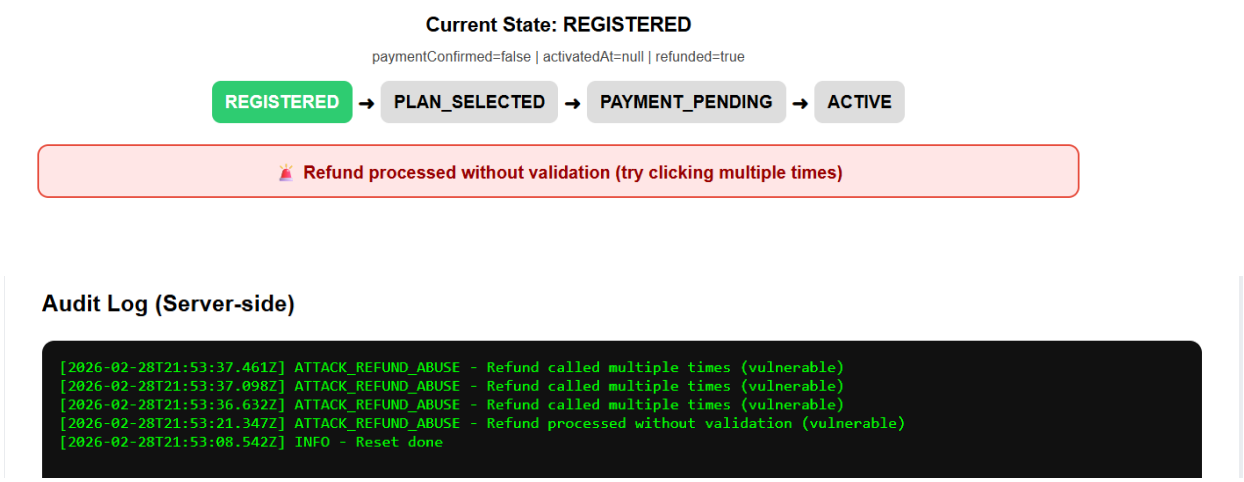

Figure 2a — Refund executed from invalid state and repeated multiple times without validation.

4. Defense and Mitigation

Why Existing Protections Failed

The server trusted client input, did not validate subscription state transitions, did not verify payments with a trusted provider, and lacked idempotency enforcement.

Prevention Mechanisms

• Enforce strict server-side state validation.

• Verify payments with trusted payment providers.

• Implement idempotency for refund operations.

• Enforce explicit state machine validation.

• Never trust client-provided data.

5. Conclusion

Business logic vulnerabilities can cause serious financial and operational risks. Proper state validation, server-side enforcement, and secure design principles are essential to prevent payment bypass and refund abuse.

6. References

OWASP Top 10 (2021): https://owasp.org/Top10/

OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/

Node.js Documentation: https://nodejs.org/

Express.js Documentation: https://expressjs.com/